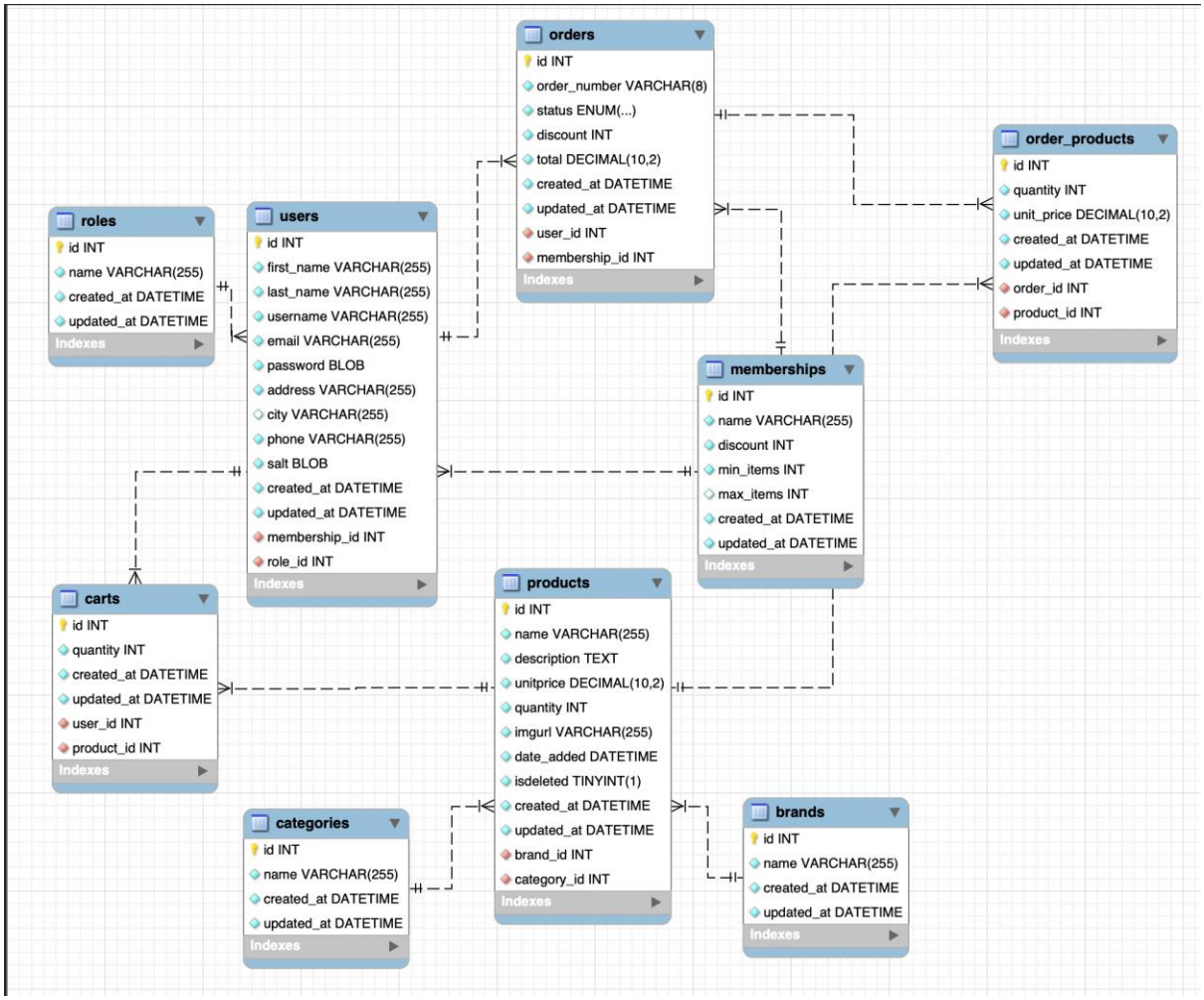
REFLECTION REPORT

Summary

This is a report about the details of the tool I chose to plan my first project exam.

Database ERD



Explaining the relationships

Brand: Brand has many Products but a Product has only one Brand, making it a one to many relationship

Cart: Cart has only one user but a User can have many Carts (lines). One to many relationship. Cart has only one product but a Product can have many Carts (lines). One to many relationship

Category: Category has many Products but a Product has only one Category, making it a one to many relationship

Membership: Membership has many User but a User has only one Membership, making it a one to many relationship. Membership has many Order but a Order has only one Membership, making it a one to many relationship

Order: Order has one User, User can have many Order, making it one to many relationship. Order has one Membership, Membership can have many Order, making it one to many relationship. Order has many OrderProduct but a OrderProducts has only one Order, making it a one to many relationship. (Membership is calculated on checkout and the minItems are never stored but rather recalculated on each new order)

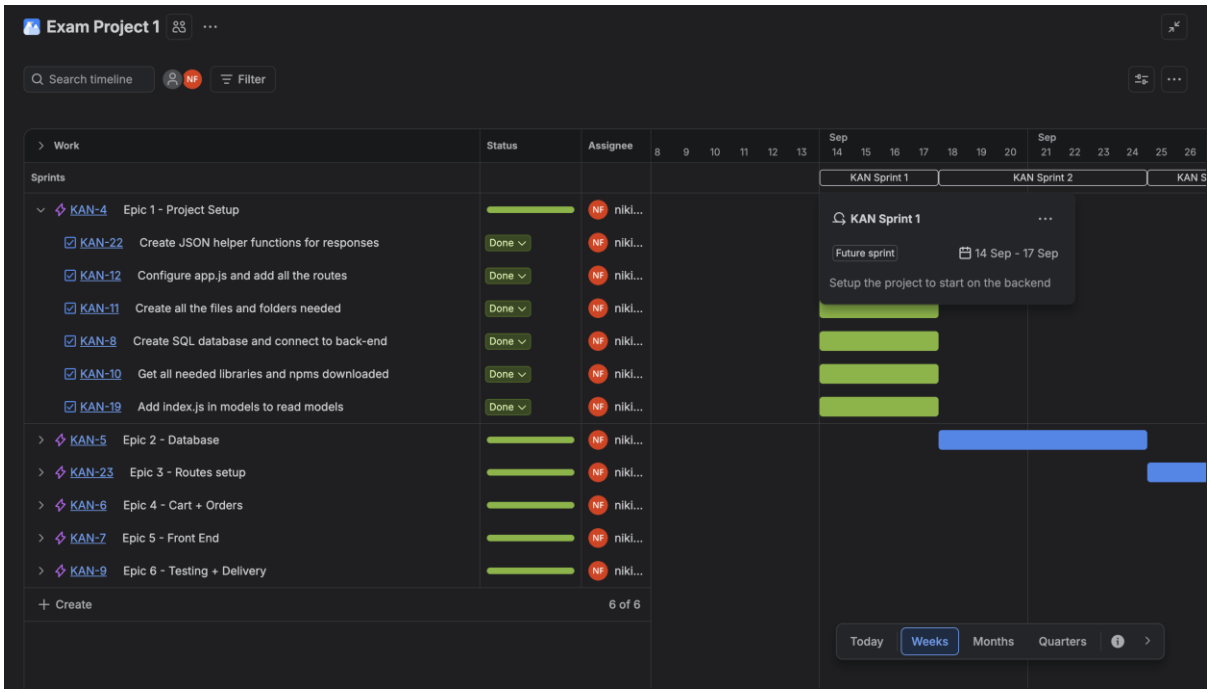
OrderProducts: OrderProduct has one Order, Order can have many OrderProduct, making it one to many relationship. OrderProduct has one Product, Product can have many OrderProduct, making it one to many relationship. (OrderProducts resolves many to many issue between Order and Product where each Product Ordered is now stored in a third table OrderProducts with checkout price and quantity)

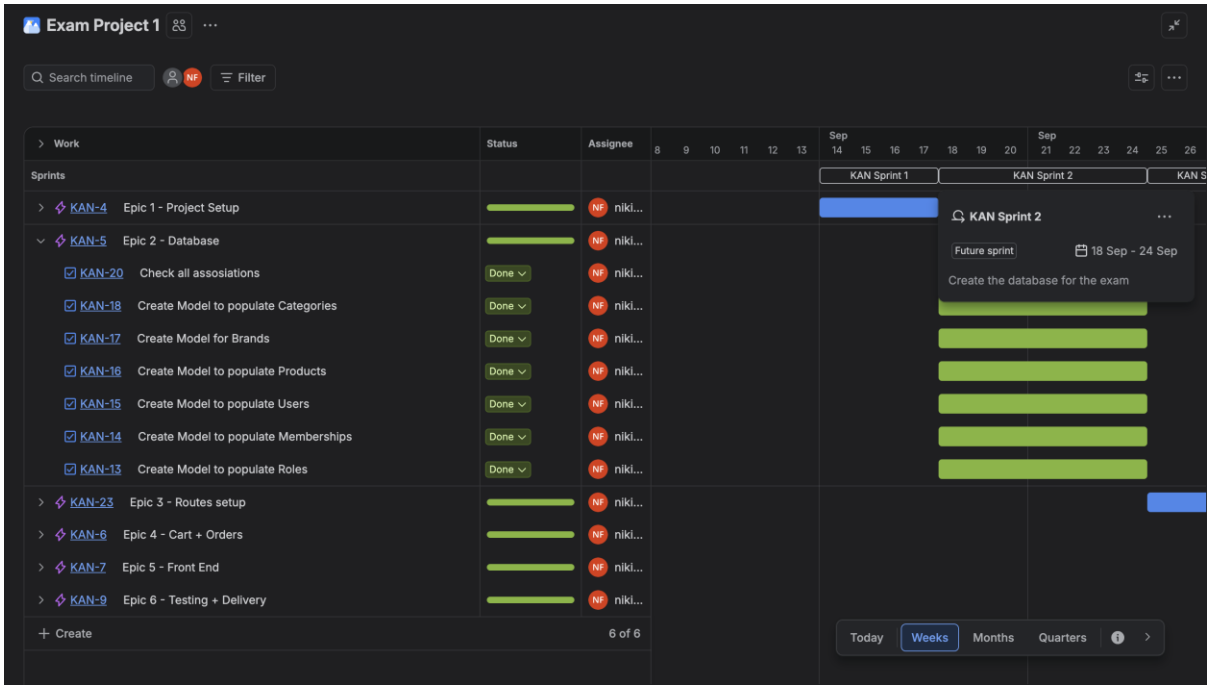
Product: Product has one Brand, Brand can have many Product, making it one to many relationship. Product has one Category, Category can have many Product, making it one to many relationship. Product has many Cart but a Cart has only one Product, making it a one to many relationship. Product has many OrderProduct but a OrderProduct has only one Product, making it a one to many relationship

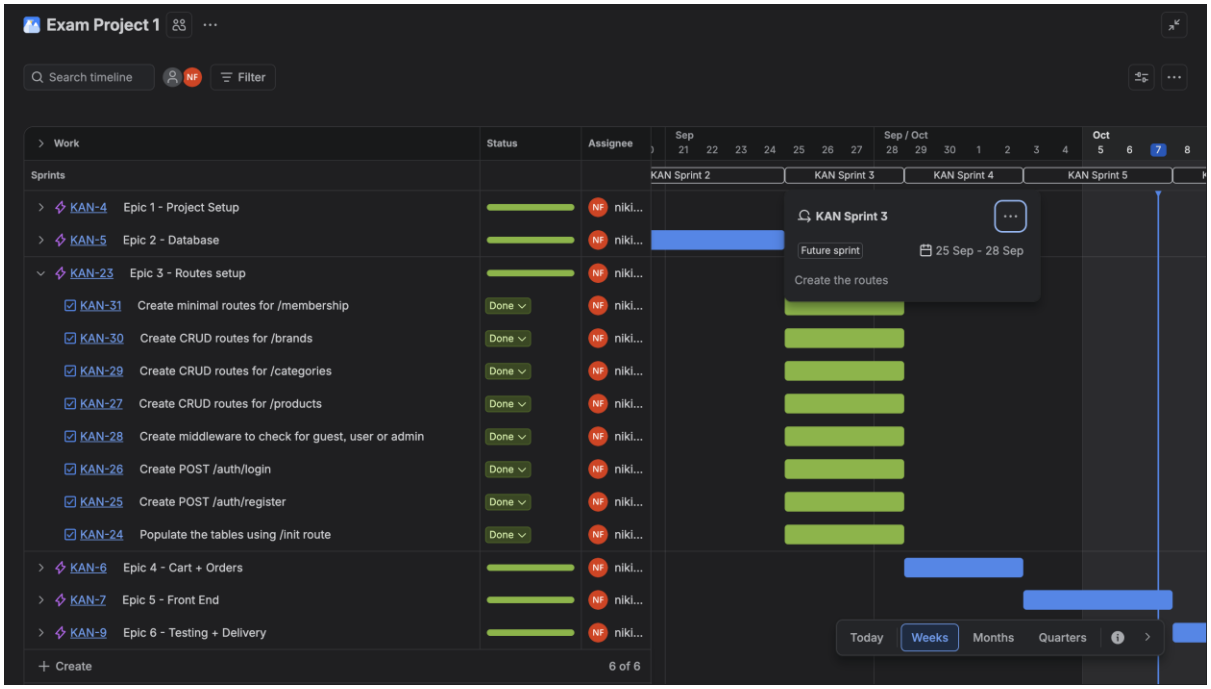
Role: Role has many Users but a User has only one Role, making it a one to many relationship

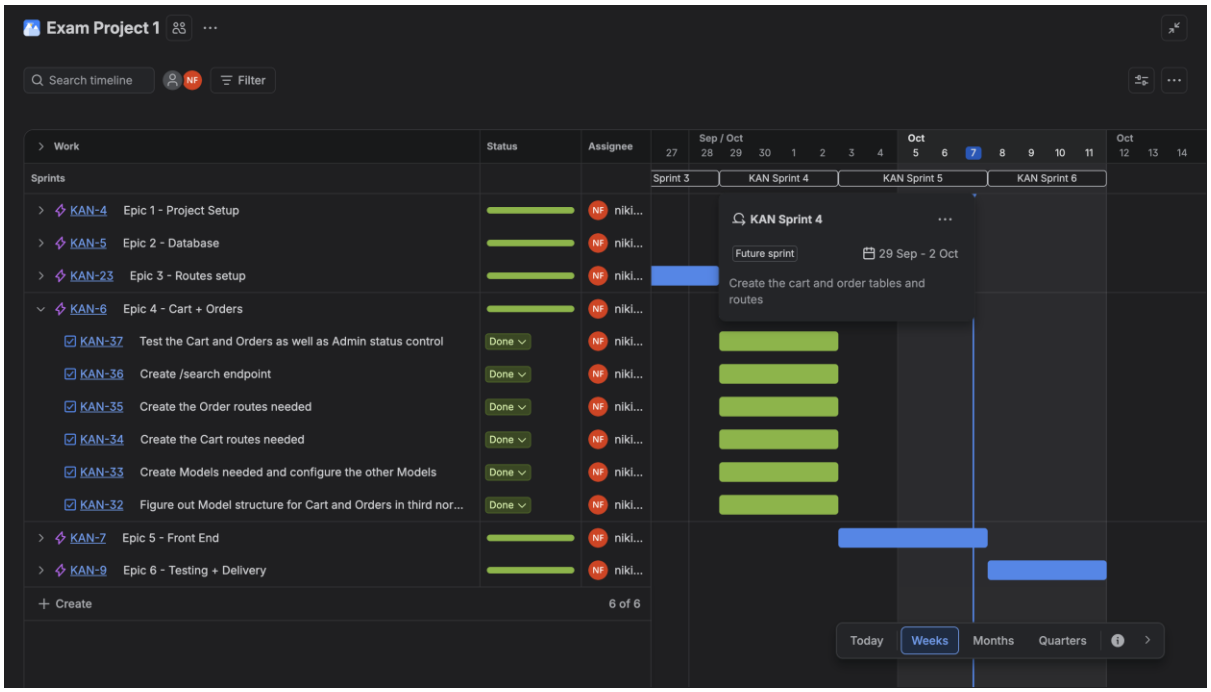
User: User has one Role, Role can have many User, making it one to many relationship. User has one Membership,Membership can have many User, making it one to many relationship. User has many Order but a Order has only one User, making it a one to many relationship. User has many Cart but a Cart has only one User, making it a one to many relationship

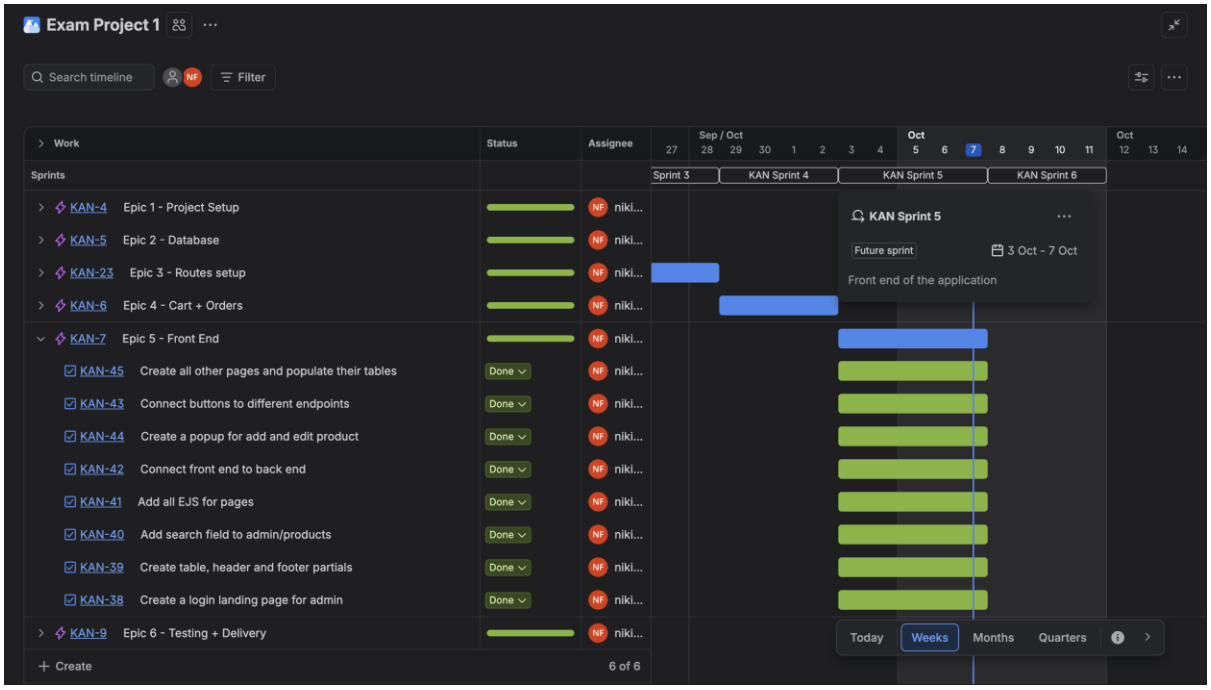
Jira Epics and Sprints

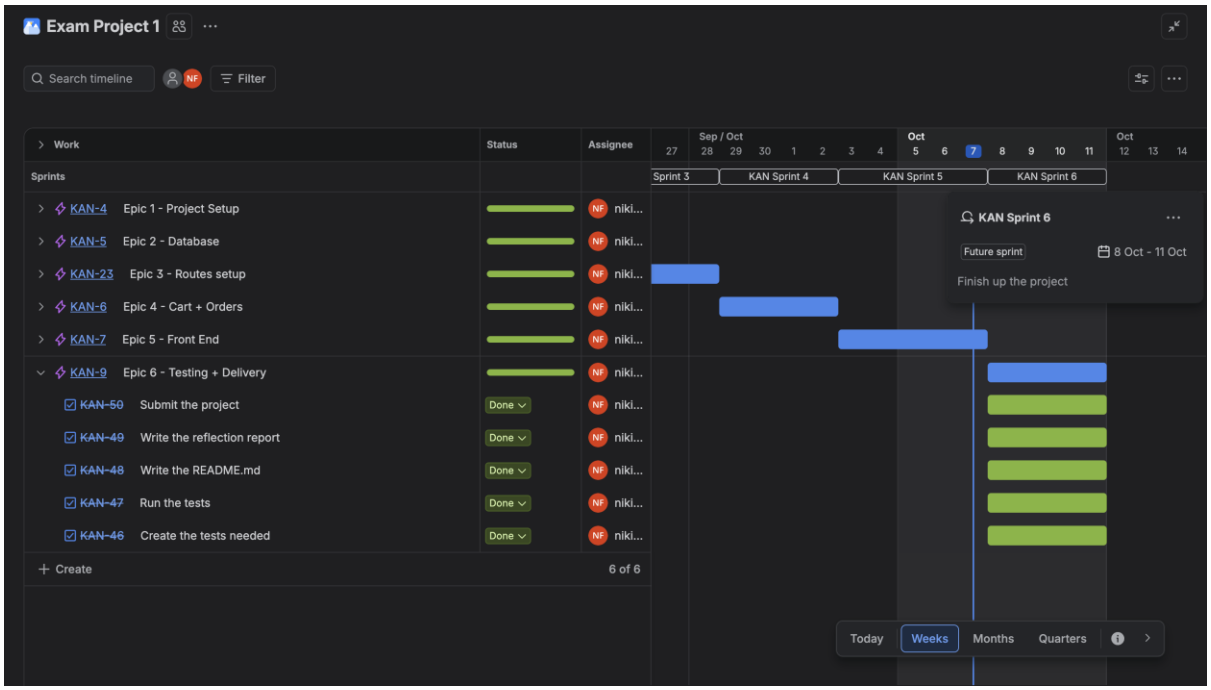





Exam Project 1
Search timeline
Filter
Work
Status
Assignee
Sep
Sep / Oct
Oct
Sprints
KAN Sprint 2
KAN Sprint 3
KAN Sprint 4
KAN Sprint 5
KAN-4 Epic 1 - Project Setup
KAN-5 Epic 2 - Database
KAN-23 Epic 3 - Routes setup
KAN-31 Create minimal routes for /membership
KAN-30 Create CRUD routes for /brands
KAN-29 Create CRUD routes for /categories
KAN-27 Create CRUD routes for /products
KAN-28 Create middleware to check for guest, user or admin
KAN-26 Create POST /auth/login
KAN-25 Create POST /auth/register
KAN-24 Populate the tables using /init route
KAN-6 Epic 4 - Cart + Orders
KAN-7 Epic 5 - Front End
KAN-9 Epic 6 - Testing + Delivery
Done
niki...
KAN Sprint 3
Future sprint
25 Sep - 28 Sep
Create the routes
Today
Weeks
Months
Quarters
Create
6 of 6


Exam Project 1
Search timeline
Filter
Work
Status
Assignee
Sep / Oct
Oct
Sprints
Sprint 3
KAN Sprint 4
KAN Sprint 5
KAN Sprint 6
KAN-4 Epic 1 - Project Setup
KAN-5 Epic 2 - Database
KAN-23 Epic 3 - Routes setup
KAN-6 Epic 4 - Cart + Orders
KAN-37 Test the Cart and Orders as well as Admin status control
KAN-36 Create /search endpoint
KAN-35 Create the Order routes needed
KAN-34 Create the Cart routes needed
KAN-33 Create Models needed and configure the other Models
KAN-32 Figure out Model structure for Cart and Orders in third nor...
KAN-7 Epic 5 - Front End
KAN-9 Epic 6 - Testing + Delivery
Done
niki...
KAN Sprint 4
Future sprint
29 Sep - 2 Oct
Create the cart and order tables and routes
Today
Weeks
Months
Quarters
Create
6 of 6





Project Reflection

-Progression of the project

During this project I decided not to jump back and forth on the instructions but rather work one page with criteria's after the other. There was too much information at first, therefor I decided to figure out the Models for the Database and their relationships. I looked into different ways of fetching and populating tested JSON from an API call. This was a completely new concept to me that I really enjoyed. This is where I got the hang of transactions and bulkCreate in sequilize. I created the the needed enpoints

and populated them as per task, isDeleted is boolean for soft delete. Index.js model file has underscore true so that the createdAt and updatedAt become the right format as well as decimalNumbers dont end up as strings. This is where I moved to the next page in the content table and found out that all my JSEND would be impossible to recreate into the needed responses, so I have created basic helper functions for both error messages and success. Now I needed to create a few middlewares to check for users/guest or admin and their JWT token. HasRole checks if there is a token or not, allowing guest and admin control. IsAuth checks for logged in users token and decodes it. IsAdmin only checks for role of isAuth user. Creating CRUD for both brand and categories endpoint was basic copy paste after I did the product endpoint. Most of services and tables word on same logic so it was always hard to get that initial code but thereafter always came a breakthrough. Products needed to be fetched using SQL query, that took some time but turned out great. Membership has only two endpoints, one for getting memeberships and one for updating the discount of the membership and nothing more. I finished up the Order endpoints and most of frontend before touching the cart logic. I tried to recreate the video shown as an example. You can only login as admin, or else you have no access to certain endpoints, there is no front end for registering users. I followed the url path in the video /admin/... for most similarity. It did not look like the search worked through query, even though the search was displayed in the URL path. I could have created a service and endpoint where I dismantle the query and used it for search but I chose to stick with the SQL query from /products since it has same logic for fetching products based on the role. Testin was just some backend login, then send a request to the endpoints and save the ids for reuse in the next test.

-Challenges faced

Biggest issue I had was with the cart and orders, I just would not touch the cart logic until I was almost done with the front-end. Since everything happens locally and the db is so small, there is no real issue with making many fetch requests, but I just could not figure out how to create service methods that would do all in one. I really struggled with the ideas behind the fetching of the cart items and general logic behind a cart with orders and their ordered Products. On one hand I believe admins should not be allowed to have carts since they are the ones who can edit the membership discount, but I left it be for all users since this is an exam. I ended up putting roles into same user endpoint in front end, admin can delete users but not admins. The JSON responses have been a nightmare for the order details, I used to get OrderProduct in each section of cart item, making the whole JSON a nested multiple level object. I figured you can map though it and create your own answers for easier readings. On the front end I was unsure whether to put multiple forms inside a single view, and how to handle errors when I have two Express applications talking to each other. I ended up creating the most basic control I could: a confirm

box for any change, buttons that stay enabled rather than being disabled, and no success messages, since the video did not have any and it felt like overkill. The membership being recalculated after the order was created was also something I did not understand at first. Why would a user not simply have the value saved in the table, rather than it being worked out after the whole order process? I ended up just counting each product in OrderedProduct by user on checkout and then updating the membership status. I check for current memebership first for the updatedAt not to fire on user just on membership change

.ENV Example

Back-end:

```
Back-end DATABASE_NAME=[your_db_name]
DB_USERNAME=[your_db_username]
DB_PASSWORD=[your_db_password]
PORT=3000
DB_PORT=3306
HOST=localhost
DIALECT=mysql
TOKEN_SECRET=[your_token_secret]
```

Front-end:

```
PORT=3001
```